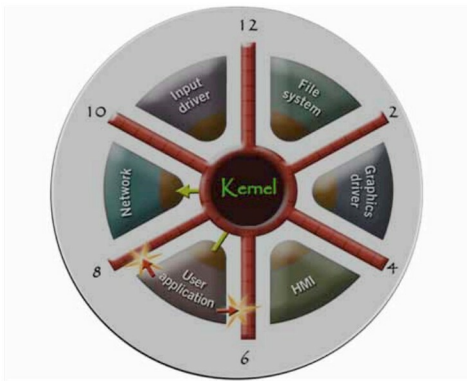




Track Time in Your Driver with Kernel Timers

This article shows how, with the help of kernel timers, resource allocation and the time-bound operation of various devices occurs without any conflicts.

Kernel timers offer a great way to keep track of time in device drivers. Instead of implementing a loop to wait for some time to expire, a timer function can provide a one-step solution. Timers are also used to poll a device at a constant interval, when the hardware can't fire an interrupt.

The kernel measures time by counting the number of ticks since the time the system has booted, and stores this count in a global *jiffies* variable. The *define HZ* controls the rate of ticks and can be modified by the user in *<asm/param.h>*.

A kernel timer is a data structure that allows the kernel to invoke a user-defined function, with user-defined arguments and user-defined time. This is defined in *linux/timer.h* and *kernel/timer.h*. In recent kernels, there are two ways to implement timers. The first is the timer API, which is used in most places, but it is less accurate and quite simple. The other is the *high-resolution timer API*, which allows us to define time in nanoseconds.

A simple timer API

The timers are implemented as a doubly linked list; the structure is shown below:

```
struct timer_list {
    struct timer_list *next;
    struct timer_list *prev;
    unsigned long expires;

    void (*function)(unsigned long);
    unsigned long data;

    int slack;
};
```

The kernel provides a driver with helper functions to declare, register and remove kernel timers. You will never